

ELK Stack

Interview Questions & Answers

Organized by topic — Elasticsearch, Logstash, Kibana, Beats & DevOps deployment

1. Introduction to the ELK Stack

Q: What is ELK?

A: ELK stands for Elasticsearch, Logstash, and Kibana. It is used to collect, process, store, search, and visualize logs.

Q: What is Elasticsearch?

A: A distributed search and analytics engine that stores data as JSON documents and enables fast searching and aggregation.

Q: What is Logstash?

A: An open-source data processing pipeline that collects data from multiple sources, processes or transforms it using filters, and sends it to destinations such as Elasticsearch.

Q: What is Kibana?

A: The visualization and analytics tool of the Elastic Stack. It reads data from Elasticsearch and lets users search logs, build dashboards and visualizations, monitor applications, and generate alerts.

Q: Why is ELK used?

A: For centralized logging, monitoring, troubleshooting, security analysis, and performance monitoring.

2. Core Concepts: Cluster, Node, Index, Document, Field

Q: What is a cluster?

A: A group of one or more Elasticsearch nodes working together and sharing the same cluster name.

Q: What is a node?

A: A single running Elasticsearch instance that stores data and participates in indexing and search operations.

Q: What is an index?

A: A logical collection of related JSON documents, similar to a database (or table) in a relational database system.

Q: What is a document?

A: A JSON object that represents a single record, such as one log entry.

Q: What is a field?

A: A key-value pair inside a document — similar to a column in a relational database.

3. Shards, Replicas & Routing

Q: What is a shard?

A: A partition of an index. Elasticsearch splits large indexes into multiple shards so data can be distributed across nodes and searches can be processed in parallel.

Q: What is a replica?

A: A copy of a primary shard that provides high availability, fault tolerance, and additional search capacity.

Q: Why is Elasticsearch so fast?

A: It uses an inverted index, which enables quick full-text searches, combined with parallel execution across multiple shards.

Q: How does Elasticsearch store data physically?

A: Documents are stored as JSON inside indices. Each index is split into primary shards distributed across nodes; each shard is stored as Lucene indexes (segments) on disk, with optional replica shards providing redundancy. Flow: Application → Filebeat/Logstash → Elasticsearch Index → Primary Shards → Lucene Segments on Disk → Search through Kibana.

Replication

Q: Why are replicas used?

A: To prevent data loss during node failures, keep the cluster available, improve search performance by distributing search requests, and increase reliability.

Q: Can writes go directly to a replica?

A: No. All writes go to the primary shard first; once the primary succeeds, it replicates the operation to its replica shards.

Q: Can searches use replicas?

A: Yes. Elasticsearch can execute searches on both primary and replica shards, which improves performance by sharing the search load.

Q: What happens if a primary shard fails?

A: Elasticsearch automatically promotes one of its replica shards to become the new primary, so the cluster keeps operating.

Changing Shard Count: Split & Shrink

Q: Can you change the number of primary shards directly?

A: No. To change the shard count you must create a new index using the Split API, the Shrink API, or by reindexing into a new index with the desired configuration.

Q: What is the Split API?

A: Creates a new index with a larger number of primary shards, spreading the data across more shards.

Q: What is the Shrink API?

A: Creates a new index with fewer primary shards by combining existing shards, reducing overhead for smaller datasets.

Q: Does Split or Shrink modify the existing index?

A: No. Both create a new index; the original remains unchanged unless you delete it or switch an alias to the new index.

Routing

Q: What is routing in Elasticsearch?

A: The mechanism Elasticsearch uses to determine which primary shard stores a document and which shard is searched for it.

Q: What is the default routing value?

A: The document's `_id`.

Q: Can we specify custom routing?

A: Yes — for example: `PUT employees/_doc/1?routing=devops`

Q: Why do we use routing?

A: To reduce the number of shards searched, improve query performance, and group related documents together.

Q: What happens if you forget the routing value when retrieving a routed document?

A: Elasticsearch looks in the shard determined by the default routing (`_id`) and may not find the document even though it exists.

Q: Does routing affect replicas?

A: Yes. Routing determines the primary shard first, and Elasticsearch automatically stores the document there and on its replica shards.

Q: What is a hot shard?

A: A shard that receives too many documents or requests relative to others, causing uneven load and reduced performance.

4. Node Roles

Q: Why do we separate node roles in production?

A: Separating roles prevents one node from handling every responsibility, improving performance, scalability, and availability. Dedicated master nodes focus on cluster management, data nodes handle storage and queries, ingest nodes process incoming documents, and coordinating nodes route client requests.

Role	Responsibility
Master Node	Manages the cluster and allocates shards
Data Node	Stores shards; uses the most disk
Ingest Node	Processes/transforms incoming documents
Coordinating Node	Routes client requests and merges search results; best for Kibana connections (or a load balancer in front of it)
ML Node	Runs machine learning jobs
Transform Node	Transforms data
Remote Cluster Client Node	Performs Cross-Cluster Search (CCS) / Cross-Cluster Replication (CCR)

Q: Can multiple Elasticsearch nodes run on one machine?

A: Yes, by assigning different HTTP ports, transport ports, data directories, and node names. This is useful for development and testing.

Q: Why don't companies use multiple nodes on one server in production?

A: Because if that single server fails, all nodes on it fail together, eliminating the benefits of high availability. Production clusters spread nodes across multiple physical or virtual machines.

Q: Why is cluster.name required?

A: Nodes with the same cluster.name discover each other and join the same Elasticsearch cluster.

Q: What is node.name?

A: It uniquely identifies an Elasticsearch node within the cluster.

Q: What does discovery.seed_hosts do?

A: It lists the addresses of other nodes that Elasticsearch contacts during cluster discovery.

Q: How can you verify that nodes joined the cluster?

A: `curl http://localhost:9200/_cat/nodes?v`

Q: How do you verify where primary and replica shards are located?

A: `curl http://localhost:9200/_cat/shards?v` — this shows which node holds each primary (p) and replica (r) shard.

5. Elasticsearch Internals: Inverted Index & Doc Values

Inverted Index

Q: What is an inverted index?

A: A data structure that maps terms to the documents containing them, so Elasticsearch can perform fast full-text searches without scanning every document.

Q: Why is it called an “inverted” index?

A: A traditional index maps document → words, while an inverted index maps word → documents — the reverse relationship.

Q: How does an inverted index improve search performance?

A: Instead of checking every document, Elasticsearch looks up the searched term in the inverted index and immediately retrieves the matching document IDs.

Q: What information does an inverted index store?

A: Terms (tokens), the document IDs where each term appears, term frequency, term positions (for phrase queries), and metadata used for scoring and highlighting.

Q: Why is an analyzer important for an inverted index?

A: The analyzer converts raw text into searchable tokens (tokenization, lowercasing, stop-word removal, optional stemming); these tokens are what get stored in the inverted index.

Doc Values

Q: What are doc values?

A: A column-oriented, on-disk data structure that stores field values by document, used for sorting, aggregations, and scripting.

Q: Why are doc values needed?

A: The inverted index is optimized for searching terms, but sorting and aggregations need quick access to field values per document — doc values provide that.

Q: Do all fields have doc values?

A: No. Most numeric, date, boolean, keyword, IP, and geo fields have doc values by default; text fields do not.

Q: What happens if doc values are disabled?

A: The field can still be searchable (if indexed), but efficient sorting, aggregations, and most script operations on it will not be available.

Q: When are doc values created?

A: Automatically at index time for field types that support them (keyword, numeric, date, boolean), unless explicitly disabled.

Q: Why are doc values stored on disk?

A: To reduce JVM heap usage; the OS file cache keeps frequently accessed data in memory, giving good performance while allowing Elasticsearch to scale.

Q: Can searching use doc values?

A: No. Full-text searching uses the inverted index; doc values are used for sorting, aggregations, and scripting.

6. Mapping

Q: What is mapping in Elasticsearch?

A: Mapping defines how fields are stored, indexed, and interpreted in an index — similar to a schema in a relational database.

Q: What is dynamic mapping?

A: Elasticsearch automatically detects new fields and their data types when documents are indexed. Enabled by default.

Q: What is explicit mapping?

A: The developer defines field types before indexing data, giving better control and preventing incorrect data types.

Q: Which mapping is recommended for production?

A: Explicit mapping, because it prevents mapping conflicts and unexpected fields.

Q: Can you change the type of an existing field?

A: No. Create a new index with the correct mapping and reindex the data.

Q: What is dynamic: false?

A: Unknown fields are stored in `_source` but not added to the mapping, so they are not indexed and cannot be searched, sorted, or aggregated.

Q: What is dynamic: strict?

A: Elasticsearch rejects any document containing fields not defined in the mapping, returning a mapping exception.

Q: Which API views the mapping?

A: GET employees/_mapping

Q: Which API adds a new field to a mapping?

A: PUT employees/_mapping

Q: Can you delete a field from a mapping?

A: No. Create a new index with the desired mapping and reindex the documents.

Coercion

Q: What is coercion?

A: Elasticsearch's ability to automatically convert compatible values to a field's mapped data type during indexing — e.g. the string "50000" is converted to the integer 50000 for a field mapped as integer.

Q: What happens if coercion is disabled?

A: Elasticsearch rejects values that don't exactly match the mapped type, even if they could otherwise be converted.

Q: Does coercion convert every value?

A: No, only values that can be safely parsed into the target type; an invalid value like "abc" for an integer field still causes an error.

7. Analyzers & Text Handling

Q: What is an analyzer?

A: A component that converts text into searchable tokens by applying a character filter, a tokenizer, and token filters.

Q: What are the three components of an analyzer?

A: Character Filter (modifies text before tokenization), Tokenizer (splits text into tokens), and Token Filter (modifies tokens after tokenization).

Q: What is the default analyzer?

A: standard

Q: Difference between an analyzer and a tokenizer?

A: An analyzer performs full text processing (including filters); a tokenizer only splits text and includes no filters.

Q: Difference between text and keyword field types?

A: text is analyzed and used for full-text search with match queries; keyword is not analyzed and used for exact matches with term queries.

Q: Which API tests an analyzer?

A: GET _analyze

Q: Can we create a custom analyzer?

A: Yes, by combining character filters, a tokenizer, and token filters.

Q: Why do we use analyzers?

A: To normalize text, improve search quality, handle case differences, remove unnecessary words, apply stemming or synonyms, and make full-text search more effective.

Q: Why does a match query work on a text field but a term query often doesn't?

A: A text field is analyzed at index time (e.g. lowercased and tokenized). A match query analyzes the search text the same way, so it matches stored tokens. A term query does not analyze the input and looks for an exact token, so it's better suited to keyword fields.

8. Search Queries

Full-Text Queries

Q: What is a full-text query?

A: A query that analyzes the search input and searches analyzed text fields.

Q: Why is match preferred for text fields?

A: Because it uses the field's analyzer to tokenize and normalize both the indexed text and the search query.

Q: When should you use term instead of match?

A: For exact-value fields such as keyword, IDs, status, dates, numeric values, or booleans.

Q: Difference between match_phrase and match?

A: match looks for matching terms in any order; match_phrase requires the terms to appear together in the specified order.

Q: Why use match_phrase_prefix?

A: To implement autocomplete or search-as-you-type functionality, where the last word may be incomplete.

Q: What is multi_match used for?

A: Searching the same query text across multiple fields.

Q: What is the advantage of combined_fields?

A: It treats multiple related fields as a single virtual field, often improving relevance.

Q: Why is simple_query_string safer than query_string?

A: It ignores invalid syntax instead of failing the entire query.

Q: What is tokenization?

A: The process of splitting text into searchable tokens.

Q: What algorithm does Elasticsearch use for relevance scoring?

A: BM25, by default.

Fuzziness

Q: What is fuzziness in Elasticsearch?

A: It allows Elasticsearch to match terms similar to the search term by tolerating a specified number of character edits, useful for handling typos.

Q: What is Levenshtein Distance?

A: The minimum number of single-character edits (insertions, deletions, substitutions, or adjacent transpositions) required to transform one string into another.

Q: What does AUTO fuzziness do?

A: It auto-selects the allowed edit distance by term length: 0–2 characters → 0 edits, 3–5 characters → 1 edit, 6+ characters → 2 edits.

Q: Difference between match with fuzziness and a fuzzy query?

A: match + fuzziness uses the field's analyzer and is best for full-text search boxes; the fuzzy query works directly on terms and is more configurable (e.g. prefix_length, max_expansions).

Q: What are prefix_length and max_expansions?

A: prefix_length is the number of initial characters that must match exactly before fuzzy matching begins; max_expansions is the maximum number of similar terms Elasticsearch will generate and evaluate, balancing accuracy and performance.

Boolean, IDs & Terms Lookup Queries

Q: Which query combines multiple queries in Elasticsearch?

A: The bool query, which combines multiple query clauses.

Q: Difference between must and filter?

A: must contributes to the relevance score; filter does not affect scoring and is optimized for exact filtering.

Q: When should you use should?

A: For optional conditions or OR logic; it can also boost the relevance score for documents matching preferred conditions.

Q: What is minimum_should_match?

A: It specifies the minimum number of should clauses that must match for a document to be returned.

Q: Can you nest bool queries?

A: Yes — nested bool queries can express complex logic such as (A AND B) OR (C AND D).

Q: What is an IDs query?

A: A term-level query that retrieves documents by their Elasticsearch document _id, without using an analyzer, and can return multiple documents.

Q: Difference between an IDs query and the GET API?

A: The GET API (GET /index/_doc/{id}) retrieves a single document by _id; the IDs query uses the Search API to retrieve one or more documents by _id and can combine with other queries.

Q: Difference between an IDs query and a term query?

A: The IDs query searches the document `_id` (a metadata field) for an exact match; a term query searches a specific field value (keyword, numeric, date, boolean, etc.) for an exact match.

Q: What is a Terms Lookup Query?

A: A query that retrieves values from a field in one document and uses those values in a terms query against another index (or the same index). It is not a JOIN — it simply reads values from one document and uses them as filter values.

Q: When should you use Terms Lookup?

A: When one document stores a list of IDs or keywords that should filter documents in another index — e.g. user favorites, permissions, allowed categories, or access-control lists.

9. Aggregations

Q: What is an aggregation?

A: A summary of data — grouping documents, calculating statistics, or analyzing trends — without returning every matching document.

Q: What are the three main types of aggregations?

A: Metrics Aggregations (sum, avg, min, max, cardinality), Bucket Aggregations (terms, range, histogram, date_histogram), and Pipeline Aggregations (avg_bucket, max_bucket, derivative, bucket_script).

Q: Difference between metrics and bucket aggregations?

A: Metrics return calculated values (e.g. average price); buckets group documents first (e.g. by category or month).

Q: Difference between stats and extended_stats?

A: stats returns count, min, max, avg, sum; extended_stats adds variance, standard deviation, and sum of squares.

Q: What is cardinality?

A: An approximate count of unique values in a field, using the HyperLogLog++ algorithm, which is memory-efficient for large datasets.

Q: When should you use pipeline aggregations?

A: When you need to calculate values based on the output of other aggregations — e.g. averages across buckets, growth rates, moving averages, or custom formulas.

10. Document Operations & the Bulk API

	PUT	POST
Behavior	Creates or replaces a document with a specified ID	Usually used when Elasticsearch generates the ID, or for update APIs
Idempotent	Yes	Not always

Q: What is doc in the Update API?

A: The partial document that should be merged into the existing document without replacing the whole document.

Q: What is doc_as_upsert?

A: It tells Elasticsearch: if the document doesn't exist, use the contents of doc to create it.

Q: Difference between doc_as_upsert and upsert?

A: doc_as_upsert reuses the same doc for both update and insert (simpler); upsert lets you specify a different document to use for the insert (more flexible).

Q: What do _seq_no and primary_term do?

A: _seq_no tracks the order of write operations, while primary_term identifies the current primary shard generation. Together they implement optimistic concurrency control, preventing stale clients from overwriting newer data in a distributed cluster.

Bulk API

Q: What is the Bulk API?

A: It performs multiple indexing, create, update, and delete operations in a single HTTP request, reducing network overhead and improving indexing performance.

Q: Difference between index and create operations?

A: index creates or replaces an existing document; create only creates and fails if the document already exists.

Q: Can one bulk request contain different operations?

A: Yes — a single request can include any combination of index, create, update, and delete operations.

Q: What does "errors": true mean in a bulk response?

A: At least one operation failed; inspect the items array to identify which operations failed and why.

Topic	Answer
Endpoint	POST / _bulk
Format	NDJSON (newline-delimited JSON)
Content-Type	application/x-ndjson
Operations	index, create, update, delete
Advantage	High-performance batch processing
Check for failures	errors field and items array
Common producers	Logstash, Beats, Kafka, custom applications
Best practice	Use moderate batch sizes, inspect failures, retry only failed items

11. Index & Data Lifecycle Management

Reindex API

Q: What is the Reindex API?

A: It copies documents from one index to another, commonly used for schema changes, index migration, or data transformation.

Q: Why do we use Reindex?

A: Because Elasticsearch does not allow changing the type of an existing field; instead we create a new index with the correct mapping and copy the data.

Q: Does Reindex move or copy data?

A: It copies data — the source index remains unchanged until you delete it.

Q: Can Reindex copy only selected documents?

A: Yes, using a query in the source section to filter which documents are copied.

Q: Can Reindex modify documents?

A: Yes, using a Painless script to add, remove, or update fields before documents are written to the destination index.

Index Templates

Q: What is an Index Template?

A: A predefined configuration that automatically applies settings, mappings, and aliases to newly created indices matching a specified index pattern.

Q: Why do we use Index Templates?

A: To avoid manually configuring every new index and to keep mappings, settings, and aliases consistent across indices — e.g. in an ELK logging environment where Logstash creates a new index daily.

Q: What are the main parts of an Index Template?

A: `index_patterns`, settings, mappings, aliases, and optionally `priority` and `composed_of` for component templates.

Q: Does an Index Template affect existing indices?

A: No — it only applies when a new index is created that matches the template's `index_patterns`.

Q: What happens if multiple templates match the same index?

A: The matching template with the highest priority is applied.

Aliases & Cross-Cluster Search

Q: What is an alias?

A: A logical name that points to one or more indices; applications use the alias instead of the physical index name.

Q: Why do we use aliases?

A: For zero-downtime migrations, searching across multiple indices, filtered views, and switching indices without changing application code.

Q: Can one alias point to multiple indices?

A: Yes, making it easy to search across all of them with a single query.

Q: What is `is_write_index`?

A: When an alias points to multiple indices, it tells Elasticsearch which index should receive write operations.

Q: What is a filtered alias?

A: An alias with a predefined query filter attached, so every search through it automatically returns only matching documents — useful for multi-tenant or department-based access.

Q: What is Cross-Cluster Search (CCS)?

A: It lets a local Elasticsearch cluster search indices located in one or more remote clusters using a single query, without copying data.

Q: Difference between CCS and CCR?

A: CCS searches remote clusters without copying data; CCR (Cross-Cluster Replication) replicates data from a leader cluster to a follower cluster for high availability or disaster recovery.

Zero-Downtime Reindex Scenario

Scenario: an application uses the `employees` index and the `phone` field was mistakenly mapped as text but needs to be keyword.

- Create a new index (e.g. `employees_v2` / `users_v2`) with the correct mapping.
- Use the Reindex API to copy documents from the old index to the new one, optionally filtering or transforming with a script.
- Verify the data in the new index.
- Move the application's alias from the old index to the new index.
- Delete the old index once it is no longer needed.

Enrich Processor & Application-Side Joins

Q: What is an Enrich Processor?

A: An ingest pipeline processor that looks up data from a reference index and adds it to incoming documents during indexing, before the document is stored.

Q: Why use an Enrich Processor instead of a JOIN?

A: Enriching data at index time removes the need for joins or lookups at search time, making queries faster and simpler.

Q: Does the Enrich Processor update existing documents automatically?

A: No — it only enriches documents as they are indexed. If reference data changes later, you must re-execute the enrich policy and reindex existing documents.

Q: What is an Application-Side Join?

A: A technique where the application retrieves related data from multiple indexes with separate queries and combines the results, instead of relying on Elasticsearch to perform SQL-style joins.

Q: Why is an application-side join commonly used?

A: Elasticsearch is optimized for document search, not relational joins; performing joins in the application avoids expensive distributed join operations and scales better.

12. Elastic Common Schema (ECS)

Q: What is ECS?

A: A standard field naming convention ensuring logs, metrics, traces, and security events use consistent field names across data sources.

Q: Why do we use ECS?

A: It provides a common schema, making it easier to search, analyze, and visualize data from multiple sources with the same queries and dashboards.

Q: Give some common ECS fields.

A: @timestamp, message, host.name, host.ip, user.name, source.ip, destination.ip, event.action, event.outcome, process.name, http.request.method.

Q: Is ECS mandatory?

A: No. Elasticsearch can store any JSON structure; ECS is optional but strongly recommended for interoperability with Elastic tools.

Q: Which Elastic products use ECS?

A: Filebeat, Metricbeat, Winlogbeat, Auditbeat, Elastic Agent, Logstash (when configured), Elastic Security (SIEM), and Kibana dashboards.

13. Logstash

Q: What are the three stages of a Logstash pipeline?

A: Input (collects data), Filter (processes and transforms data), and Output (sends data to Elasticsearch or another destination).

Q: Why is Logstash used?

A: To collect logs from multiple sources, transform them into a structured format, and send them to Elasticsearch for storage and analysis.

Q: Can Logstash work without Elasticsearch?

A: Yes — it can send data to files, Kafka, Redis, Amazon S3, databases, or the console. Elasticsearch is just the most common output.

14. Beats & X-Pack

Q: What are Beats?

A: Lightweight agents that collect logs, metrics, or other data from servers and send them to Logstash or Elasticsearch.

Q: What is Filebeat?

A: A lightweight log shipper that reads log files and forwards them to Logstash or Elasticsearch.

Q: Which Beat is used for application logs?

A: Filebeat.

Q: What is Metricbeat used for?

A: Collecting system and application performance metrics such as CPU, memory, disk, and network usage.

Q: Why use Filebeat instead of Logstash on every server?

A: Filebeat is lightweight and consumes fewer resources, efficiently shipping data from many servers to a centralized Logstash instance or directly to Elasticsearch, reducing resource usage and simplifying management.

Q: What is X-Pack?

A: The built-in feature set of the Elastic Stack providing security, monitoring, alerting, reporting, machine learning, and other advanced capabilities. In older versions it was a separate plugin; in modern releases it is integrated by default.

15. Kibana

Q: Does Kibana store data?

A: No. Kibana reads data from Elasticsearch and displays it in a user-friendly interface.

Q: What is the Discover feature?

A: It's used to search, filter, and inspect logs stored in Elasticsearch, useful for troubleshooting and log analysis.

Q: What are Kibana dashboards?

A: They combine multiple visualizations, charts, and metrics into a single screen, letting users monitor applications and infrastructure in real time.

Q: Can Kibana generate alerts?

A: Yes, based on predefined conditions such as high CPU usage, increased error rates, or service failures, notifying users via channels like email or Slack.

Q: What is Kibana Dev Tools?

A: A built-in console for executing Elasticsearch REST API commands directly, without using curl or Postman.

16. Network Ports Reference

Q: Which port does Filebeat listen on?

A: None — Filebeat does not listen on a port; it typically sends data to Logstash on port 5044 or directly to Elasticsearch on 9200.

Q: Which port does Logstash use?

A: It depends on the configured input plugin. Common ports are 5044 (Beats), 5000 (TCP/UDP), and other configurable ports such as 8080 for HTTP input.

Q: Difference between ports 9200 and 9300?

A: 9200 is the HTTP REST API used by clients, Kibana, and applications; 9300 is the internal transport protocol used for communication between Elasticsearch nodes.

Port	Service
9200	Elasticsearch REST API
9300	Elasticsearch transport (cluster communication)
5601	Kibana
5044	Logstash Beats input
5000	Logstash TCP/UDP input

17. Docker Compose & Deployment Configuration

Q: What is Docker Compose?

A: A tool for defining and running multi-container Docker applications using a YAML file.

Q: Why do we use version: "3.8" in a compose file?

A: To specify the Docker Compose file format version and the features it supports.

Q: Why is `discovery.type=single-node` required?

A: It tells Elasticsearch to run as a standalone node instead of expecting a cluster.

Q: Why disable `xpack.security.enabled` in development?

A: To simplify setup by removing authentication and TLS. In production, security should typically be enabled.

Q: What is `ES_JAVA_OPTS`?

A: It configures the JVM heap size used by Elasticsearch; proper heap sizing matters for performance and stability.

Q: Why use `depends_on`?

A: To control the startup order of containers. It does not guarantee that a dependency is fully initialized/ready.

Q: Why use `elasticsearch:9200` instead of `localhost:9200` inside Docker?

A: Containers communicate over Docker's internal network using service names as hostnames.

Q: Why mount volumes?

A: To share files between the host and the container — for persisting Elasticsearch data and allowing configuration/log files to be updated without rebuilding the image.

Q: Why does Kibana use `ELASTICSEARCH_HOSTS`?

A: It tells Kibana which Elasticsearch node to connect to for querying data and creating visualizations.

Q: Difference between a Docker image and a container?

A: An image is a read-only template containing the application and its dependencies; a container is a running instance created from that image.

Q: Why run two Elasticsearch nodes in a local setup?

A: To form a cluster, demonstrate shard replication, improve availability, and simulate a production environment.